x64 Architecture and Programming

Pavel Yosifovich

@zodiacon

zodiacon@live.com

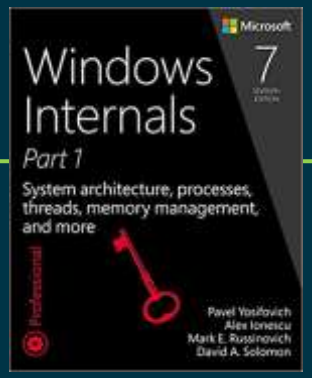
©2024 Pavel Yosifovich

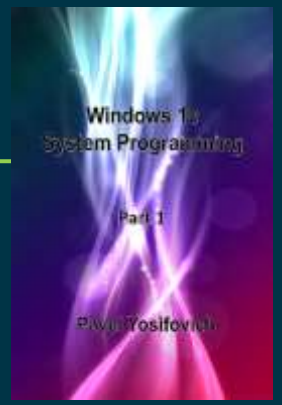
Course Details

- Goals
  - Understand the architecture and programming model of the x64 family of processors
  - Write standalone assembly as well as mixed C/C++/Assembly programs
- Target audience
  - Experienced developers and researchers
- Prerequisites
  - Basic understanding of computer systems
  - Experience writing C code

©2024 Pavel Yosifovich

About Me





- Developer, Trainer, Author, Speaker
- Book author
  - "Windows Internals 7th edition, Part 1" (co-author, 2017)
  - "Windows 10 System Programming, Part 1+2" (2020/1)
  - "Windows Kernel Programming, 2nd ed." (2023)
  - "Windows Native API programming" (WIP)
- *Pluralsight* and *Pentester Academy* course author
- Founder at https://training.trainsec.net
- Author of several open-source tools (http://github.com/zodiacon)
- Website: http://scorpiosoftware.net
- Youtube: https://youtube.com/@zodiacon

©2024 Pavel Yosifovich

Why Learn Assembly?

- Understand how computer systems work on a deep level
- Understand disassembled code for research, debugging, and/or reverse engineering
- Write optimized code when needed
- Build better software (regardless of programming language used)
- Satisfy curiosity / interest

©2024 Pavel Yosifovich

Contents

Module 0: Software Environment

Module 1: Basic Architecture

Module 2: Assembly Programming I

Module 3: Assembly Programming II

Module 4: Advanced Architecture

Module 5: Virtual Page Translation

Module 6: Miscellaneous Topics

©2024 Pavel Yosifovich

Resources

- Books
  - Windows 64-bit Assembly Language Programming Quick Start / Robert Dunne
  - Assembly Language for x86 Processors / 8th ed. / Kip Irvine
  - Moden x86 Assembly Language Programming / 3rd ed / Daniel Kusswurm
  - The Art of 64-bit Assembly Language (Vol 1) / Randall Hyde
  - x86 Instruction Set Architecture / 5th ed. / Tom Hanley
- Official Intel and AMD manuals
  - Intel: Intel® 64 and IA-32 Architectures Software Developer Manuals
  - AMD: AMD64 Architecture Programmer's Manual, Volumes 1-5
- Web
  - Handy instruction reference : x86 and amd64 instruction reference
  - Another one: https://c9x.me/x86/
  - And another: x86 instruction listings - Wikipedia
  - MASM64 SDK: https://github.com/PawelWorzala666/masm64-SDK

©2024 Pavel Yosifovich

Software Environment

Module 0

©2024 Pavel Yosifovich

Agenda

- Software and Tools
- Simple Examples
- Debugging

©2024 Pavel Yosifovich

Software and Tools

- Windows 10/11 x64
  - Windows 7/8.1 would work as well
- Visual Studio 2022 (any SKU)
  - Free “community edition” works
  - Include “Desktop C++ Development” workload
  - MASM is installed as part of that workload
- NASM assembler (https://nasm.us/)
- Optionally, another text editor (e.g., VS Code, Notepad++)

©2024 Pavel Yosifovich

Compile a Standalone Assembly Program

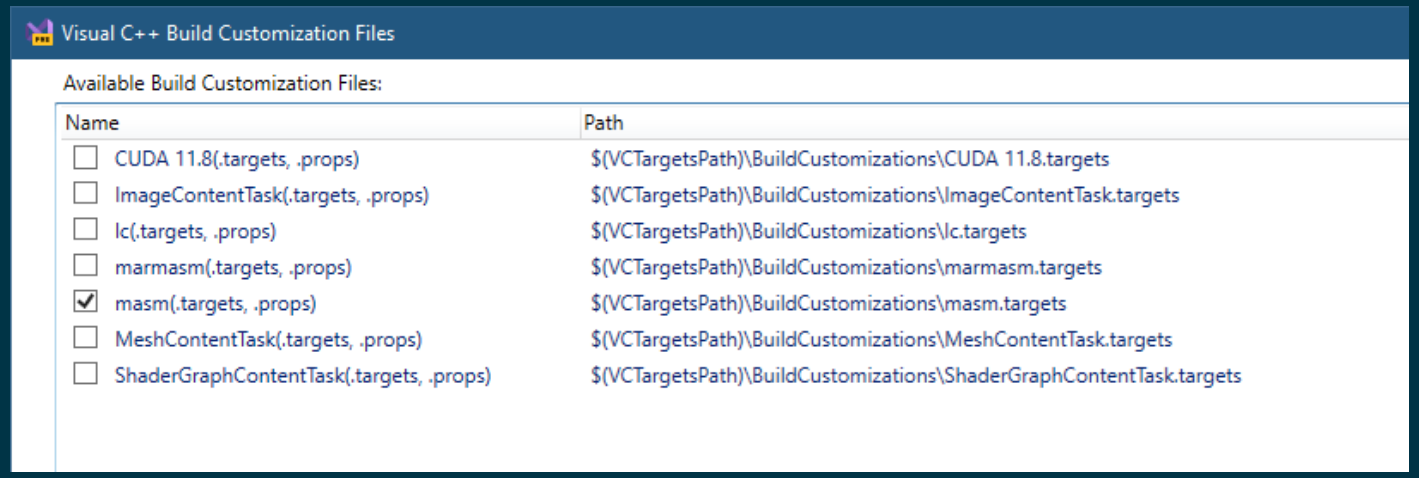
- Simplest possible program
- Open Developer Command Window and navigate to where the source file is (*hello.asm* in this example)
  - Run
    `ml64 hello.asm /link /subsystem:console /entry:main`

```
; hello.asm
.code

main proc
    ret
main endp

end
```

©2024 Pavel Yosifovich

Compiling with Visual Studio

- Assembly files in a C++ program can be compiled with Visual Studio as well
  - Start with any C++ project (e.g., Console C++ application)
  - Right-click the project's node in Solution Explorer to navigate to *Build Dependencies / Build Customizations*
  - Check "masm"



©2024 Pavel Yosifovich

Combine Assembly with C/C++

- Normal Console C++ application

Hello.cpp

```
#include <stdio.h>

extern "C" int GetMeaningOfLife();

int main() {
    int n = GetMeaningOfLife();
    printf("The meaning of life is %d\n", n);
    return n;
}
```

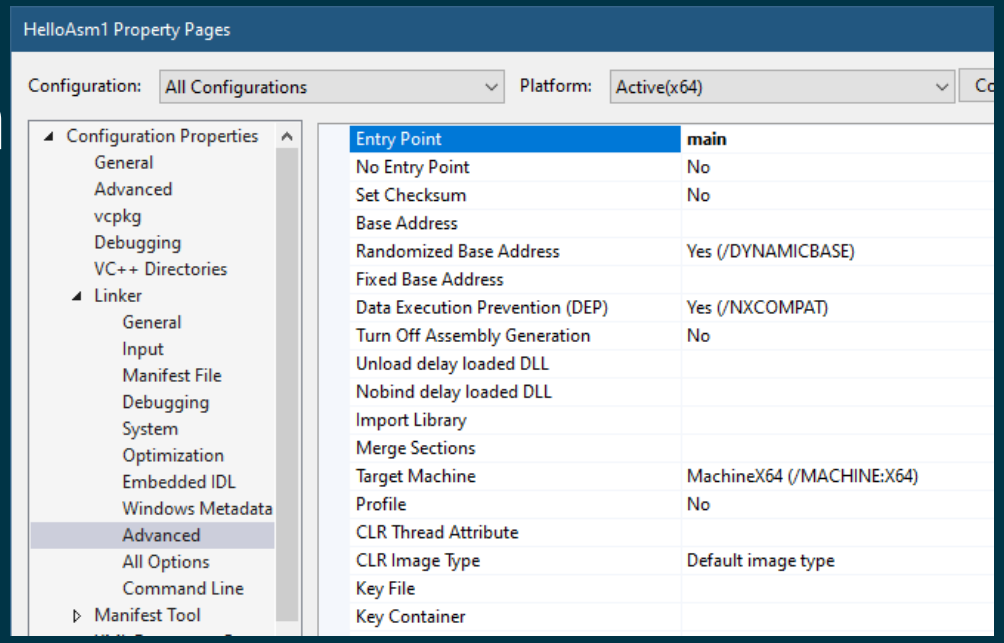
Meaning.asm

```
.code

GetMeaningOfLife proc
    mov eax, 42
    ret
GetMeaningOfLife endp

end
```

©2024 Pavel Yosifovich

Stand Alone Assembly with Visual Studio

- Start with a C++ Console application
  - Remove CPP file
- Or start with a Windows/Console empty application (“Windows Desktop Wizard”)
- Enable MASM build customization
- Specify actual entry point in Linker settings
  - Must be a procedure in assembly



©2024 Pavel Yosifovich

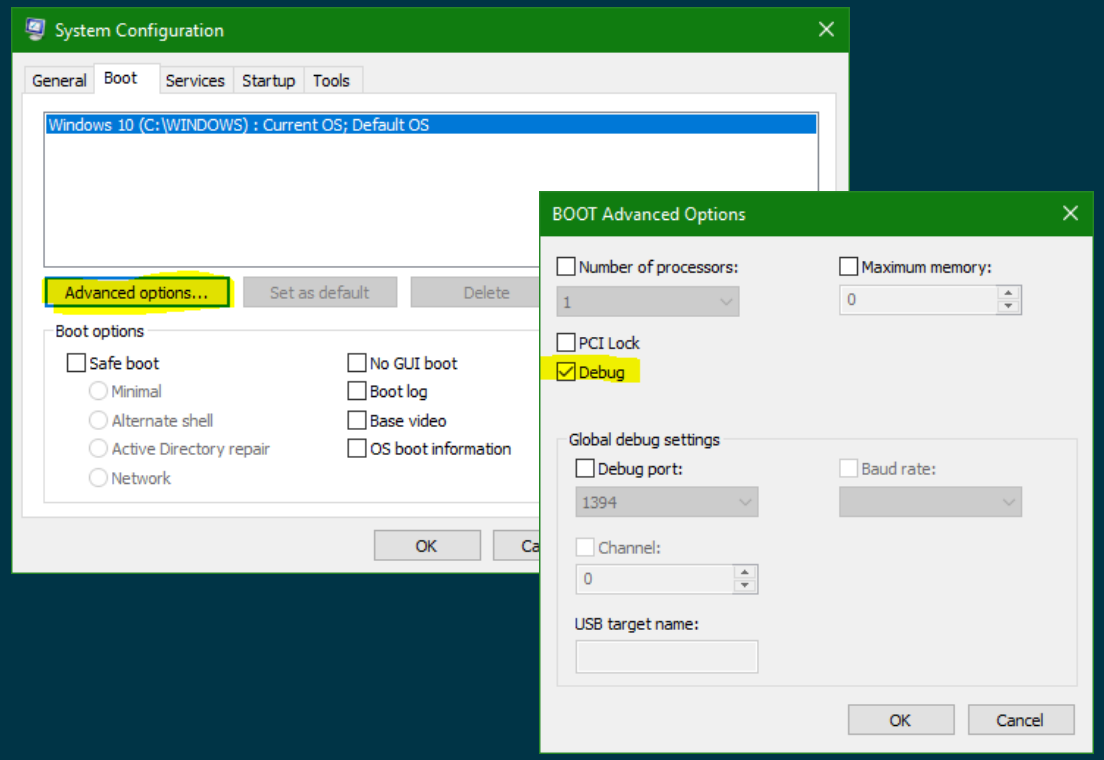
Debugging

- The Visual Studio debugger can be used to debug stand-alone or hybrid assembly applications
  - Breakpoints can be set in assembly source
  - Assembly window as well
    - Debug / Windows / Assembly to view assembly
  - Use Watch window to view registers values
    - Or open *Registers* window (Debug / Windows / Registers)
- Other debuggers work as well
  - E.g., WinDbg, x64Dbg

©2024 Pavel Yosifovich

Local Kernel Debugging

- Configure machine for kernel debugging
  - Run *MsConfig.exe* or use *bcdedit.exe* on the command line
    - `Bcdedit.exe -debug on`
  - Reboot
- Launch *WinDbg* elevated
  - File | Kernel Debug… | Local



©2024 Pavel Yosifovich

Remote Kernel Debugging

- Target machine
  - Configure for debugging as before
  - Select a communication medium
    - ***MsConfig.exe*** or ***bcdedit.exe /dbgsettings***
    - Serial, USB, Network (Windows 8+)
- Host machine
  - File | Kernel Debug…
  - Select configured communication medium
- If target is a virtual machine
  - Can expose a VM COM port as a host named pipe
  - Or use network if host and target are Windows 8+

©2024 Pavel Yosifovich

Kernel Debugging (Network)

- Target
  - Elevated command window
  - `bcdedit /dbgsettings net hostip:<ip> port:<port> [key:<key>]`
  - If key is not provided, a random key is generated
  - Restart the system
- Host (debugger)
  - Select network debugging
  - Set same port and key

©2024 Pavel Yosifovich

Local Kernel Debugging with *LiveKd*

- *LiveKd* is a Sysinternals tool
- Copy *Livekd.exe* to the directory where the classic WinDbg is located
  - Typically, c:\Program Files (x86)\Windows Kits\10\Debuggers\x64
- Open an elevated command window and navigate to the above directory
- Use the command: `LiveKd -w`
- *LiveKd* works by presenting a fictitious dump file to the debugger
  - Data may become stale due to caching
  - Quit WinDbg and launch *LiveKd* again

©2024 Pavel Yosifovich

Basic Architecture

Module 1

©2024 Pavel Yosifovich

Agenda

- Brief Historical Perspective
- Data Types
- Registers
- Numbering Systems
- Memory Addressing
- Some Basic Instructions
- Summary

©2024 Pavel Yosifovich

Historical Perspective (1)

- 80386 was First 32-bit Intel processor (1985)
  - 32-bit registers and addresses, optional flat memory model, paging
  - IA32 or x86
- Pentium (P5 microarchitecture) introduced in 1993
  - Code and data caches, MMX, dual instructions pipeline
- Pentium Pro (P6) (1995), Pentium 2 (1997), Pentium 3 (1999)
  - Out of order execution, SSE (Streaming SIMD)
- Core microarchitecture (2006)
  - SSSE3, SSE4.1, multi-core

©2024 Pavel Yosifovich

Historical Perspective (2)

- Nehalem microarchitecture (2008)
  - SSE4.2, string processing with 128-bit wide operands
  - First gen Core-i3, Core-i5, Core-i7
- Sandy Bridge microarchitecture (2011)
  - AVX (floating point with 256-bit)
  - 2nd and 3rd generation Core-i3, Core-i5, Core-i7
- Haswell microarchitecture (2013)
  - AVX2, FMA instructions, new instructions
  - 4th generation Core-i3, Core-i5, Core-i7

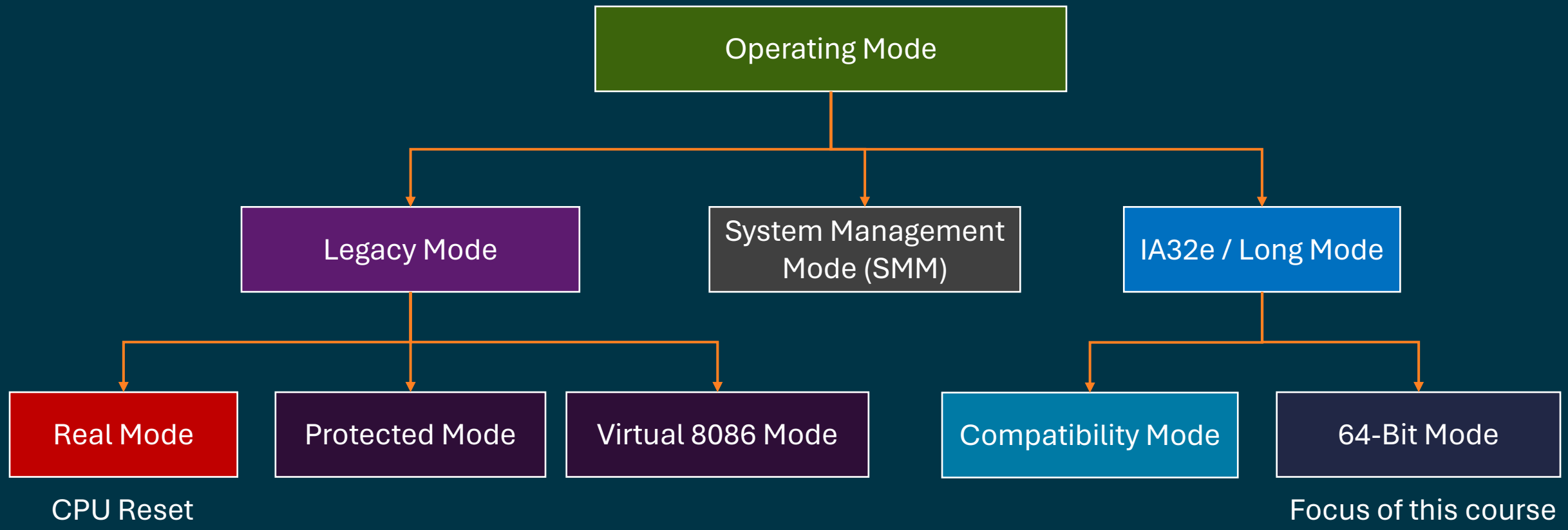
©2024 Pavel Yosifovich

Historical Perspective (3)

- AMD introduced Opteron processors in 2003
  - 64-bit (IA32e or x86-64)
  - Intel followed suite in 2004
- K10 microarchitecture (AMD) (2007)
  - SSE4a
- Bulldozer microarchitecture (AMD, 2011)
  - SSSE3, SSE4.1, SSE4.2, SSE4a, AVX, FMA4
- Newer processors (Intel and AMD)
  - AVX2, AVX-512

©2024 Pavel Yosifovich

Operating Modes



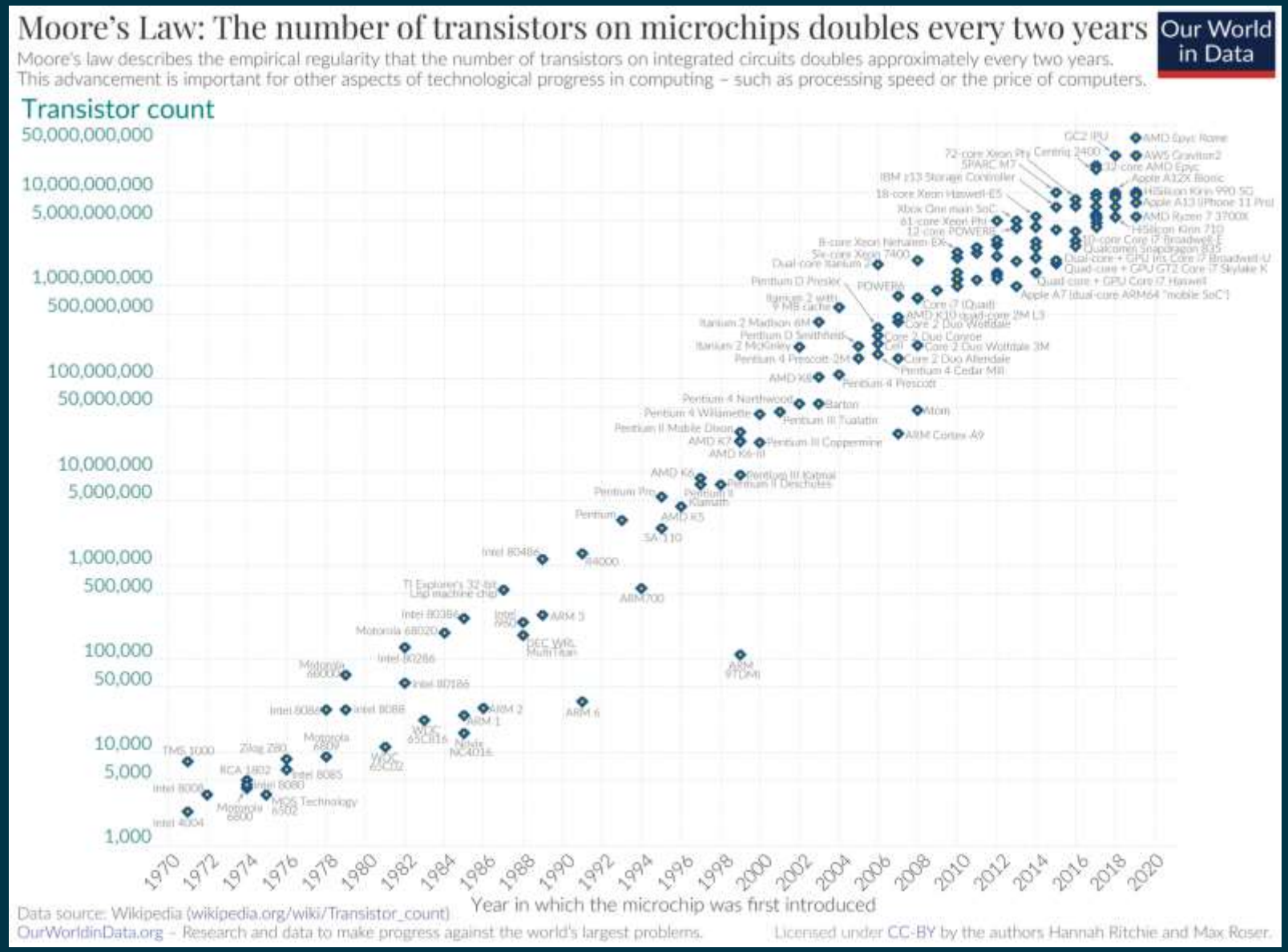
©2024 Pavel Yosifovich

Instruction Set Philosophies

- Complex Instruction Set Computing (CISC)
  - Handle many different types of operations
  - Complicated hardware runs more slowly
- Reduced Instruction Set Computing (RISC)
  - Simpler, smaller, instruction set makes it easier to build fast hardware
  - Let software do the complicated operations by composing simpler ones
- Modern reality is somewhere between the two
- x86: CISC, ARM: RISC, RISC-V: RISC

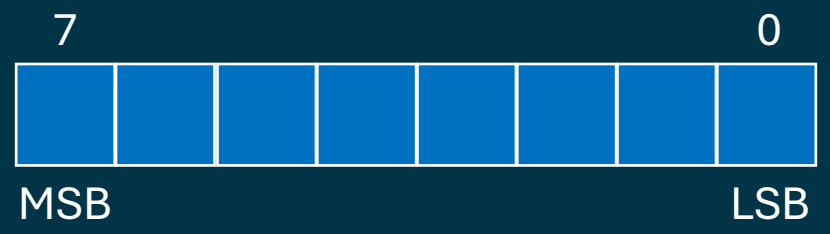
©2024 Pavel Yosifovich

Moore’s Law and Transistor Count



Fundamental Terms

- Bit
  - Smallest unit of information: 0 or 1
- Nibble (not used very often)
  - 4 bits
- Byte
  - 8 bits
  - Smallest addressable unit
- MSB (Most Significant Bit/Byte)
- LSB (Least Significant Bit/Byte)



©2024 Pavel Yosifovich

Data Types

Data type	Size (bytes)	Example usage
Byte	1 (8 bits)	ASCII characters, small integers
Word	2 (16 bits)	UTF-16 characters, integers
Doubleword	4 (32 bits)	Integers, single precision floating point
Quadword	8 (64 bits)	Integers, double-precision floating point, memory address
Octaword (Double quadword)	16 (128 bits)	Integers, packed integers, packed floating point

- These types can mostly be described by C/C++ data types
  - `char`, `short`, `long`, `long long` (and their `unsinged` variants)
  - 128-bit data type is not currently defined in C/C++
    - A structure can be used instead

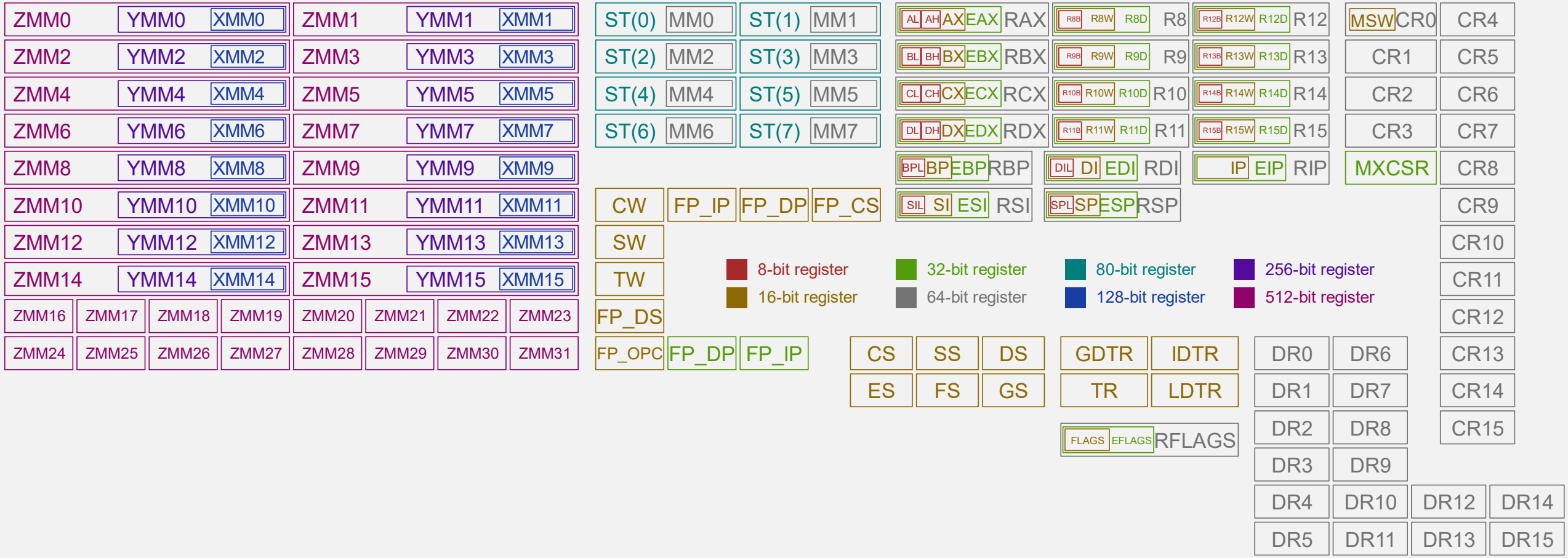
©2024 Pavel Yosifovich

x64 Processor Registers

- General purpose registers
- Instruction Pointer
- Flags
- Floating point and SIMD
- Other registers

©2024 Pavel Yosifovich

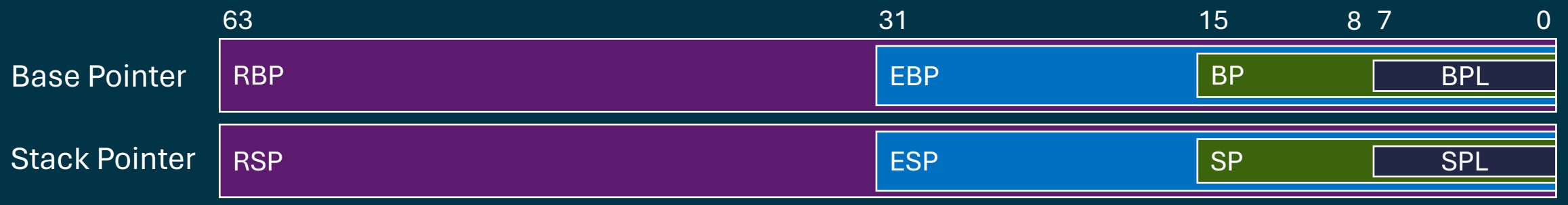
x86/x64 Registers Overview



General Purpose Registers (1)

©2024 Pavel Yosifovich

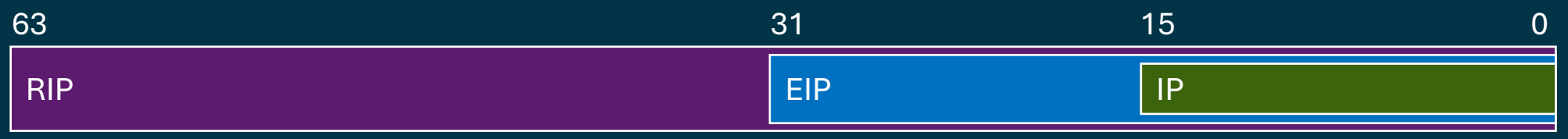
General Purpose Registers (2)



- The Stack pointer register is used implicitly with PUSH and POP instructions
  - As well as CALL and RET

©2024 Pavel Yosifovich

Instruction Pointer Register



- Used (with the CS register) to locate the next instruction
- CS is not used in 64-bit mode for base addressing
  - Base address is always zero

©2024 Pavel Yosifovich

Flags Register

Bit	Name	Description	R/W
0	CF	Carry Flag	R/W
2	PF	Parity Flag	R/W
4	AF	Auxiliary Carry Flag	R/W
6	ZF	Zero Flag (1=zero)	R/W
7	SF	Sign Flag (1=negative)	R/W
10	DF	Direction Flag	R/W
11	OF	Overflow Flag	R/W

- Other flags described later

©2024 Pavel Yosifovich

Standard Flags (1)

- Carry Flag (CF)
  - Set to 1 if the last addition results in carry, or the last subtraction results in a borrow
  - Increment and decrement instructions do not affect the CF
  - Bit shift and rotate instructions shift operand bits into the Carry
  - Logical instructions (e.g., AND, OR, XOR) clear the CF
  - CF can be set or cleared explicitly (STC and CLC), and complemented with CMC

©2024 Pavel Yosifovich

Standard Flags (2)

- Parity Flag (PF)
  - Set to 1 if the number of “one” bits in the least significant byte is even (for certain operations)
- Auxiliary Carry Flag (AF)
  - Set to 1 if an addition/subtraction of Binary Coded Decimal (BCD) operation results in a carry/borrow out of bit 3
- Zero Flag (ZF)
  - Set if the last arithmetic operation resulted in a value of zero

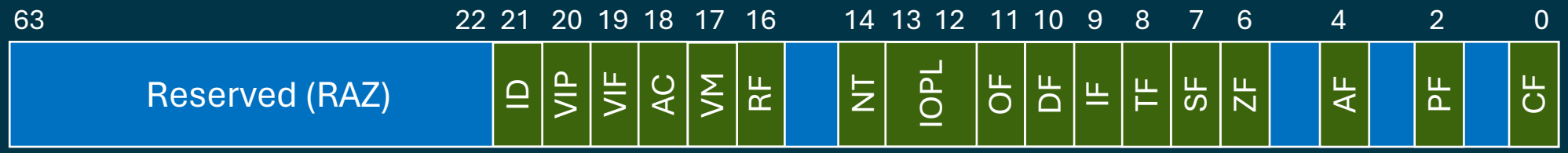
©2024 Pavel Yosifovich

Standard Flags (3)

- Sign Flag (SF)
  - Set to 1 if the last arithmetic operation resulted in a negative value (most significant bit is 1)
- Direction Flag (DF)
  - Determines the order in which strings are processed
  - 1 indicates decrementing the data pointer, while 0 indicates incrementing
  - Can be set with `STD` and cleared with `CLD`
- Overflow Flag (OF)
  - Set to 1 if the last signed integer operation differs from the signs of both source operands

©2024 Pavel Yosifovich

RFLAGS



Bits	Name	Description	Access Type
63:22	Reserved		
21	ID	ID Flag (support CPUID)	R/W
20	VIP	Virtual Interrupt Pending	R/W
19	VIF	Virtual Interrupt Flag	R/W
18	AC	Alignment Check	R/W
17	VM	Virtual-8086 Mode	R/W
16	RF	Resume Flag	R/W
14	NT	Nested Task	R/W
13:12	IOPL	I/O Privilege Level	R/W
9	IF	Interrupt Flag	R/W
8	TF	Trap Flag	R/W

©2024 Pavel Yosifovich

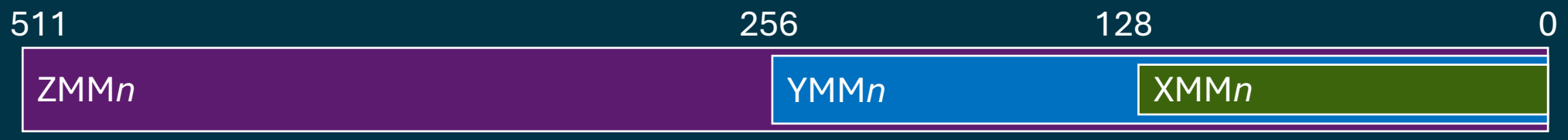
Implicit Use of General-Purpose Registers

Names	Description	Implicit Uses
AL, AX, EAX, RAX	Accumulator	• Operand for decimal arithmetic, multiply, divide, string, compare-exchange, table translation, I/O • Special encoding for ADD, XOR and MOV instructions • Used with rDX to hold double-precision operands • CPUID processor feature information
BL, BX, EBX, RBX	Base	• Address generation in 16-bit code • Memory address for XLAT instruction • CPUID processor feature information
CL, CX, ECX, RCX	Count	• Bit index for shift and rotate instructions • Iteration count for loop and repeat string instructions • Jump conditional if zero • CPUID processor feature information
DL, DX, EDX, RDX	Data	• Operand for multiply and divide instructions • Port number for I/O instructions • Used with rAX to hold double precision operands • CPUID processor feature information
SI, ESI, RSI	Source Index	• Memory address of source operand for string instructions • Memory index for 16-bit addresses
DI, EDI, RDI	Destination Index	• Memory address of destination operand for string instructions • Memory index for 16-bit addresses
R11		• Holds value of RFLAGS on SYSCALL / SYSRET

©2024 Pavel Yosifovich

Floating Point and SIMD Registers

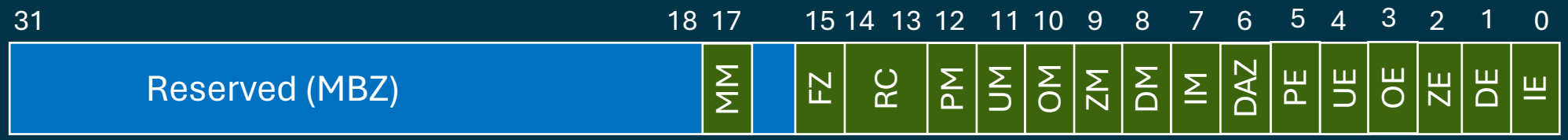
- SSE, AVX/2, AVX-512 extensions
- `XMM0` to `XMM31` (each 128 bit)
- `YMM0` to `YMM31` (each 256 bit, overlapping with XMM)
- `ZMM0` to `ZMM31` (each 512 bit, overlapping with YMM)



©2024 Pavel Yosifovich

MXCSR

- *Media eXtension Control* and *Status Register*



Bits	Name	Description	Reset value
17	MM	Misaligned Exception Mask	0
15	FZ	Flush to Zero for Masked Unerflow	0
14:13	RC	Floating Point Rounding Control	00
12	PM	Precision Exception Mask	1
11	UM	Underflow Exception Mask	1
10	OM	Overflow Exception Mask	1
9	ZM	Zero-Divide Exception Mask	1
8	DM	Denormalized-Operand Exception Mask	1
7	IM	Invalid Operation Exception Mask	1

©2024 Pavel Yosifovich

MXCSR (cont.)

Bits	Name	Description	Reset value
6	DAZ	Denormals are Zeros	0
5	PE	Precision Exception	0
4	UE	Underflow Exception	0
3	OE	Overflow Exception	0
2	ZE	Zero-Divide Exception	0
1	DE	Denormalized-Operand Exception	0
0	IE	Invalid-Operand Exception	0

- Load from memory: `XRSTOR`, `XRSTORS`, `FXRSTOR`, `LDMXCR`
- Store: `XSAVE`, `XSAVEOPT`, `XSAVEC`, `XSAVES`, `FXSAVE`, `STMXCSR`

©2024 Pavel Yosifovich

Integer Number Representation

- Unsigned numbers
  - MASM: `byte` (1), `word` (2), `dword` (4), `qword` (8), `oword` (16)
- Signed numbers
  - Represented with two's complement

Type	Size (bits)	Unsigned range	Signed range
Byte	8	0 to 255	-128 to +127
Word	16	0 to 65535	-32768 to +32767
Dword (double word)	32	0 to 4294967295	-2147483648 to +2147483647
Qword (quad word)	64	$0\ to\ 2^{64}-1$	$-2^{63} to +2^{63}-1$
Oword (octaword, double quad word)	128	$0\ to\ 2^{128}-1$	$-2^{127} to +2^{127}-1$

©2024 Pavel Yosifovich

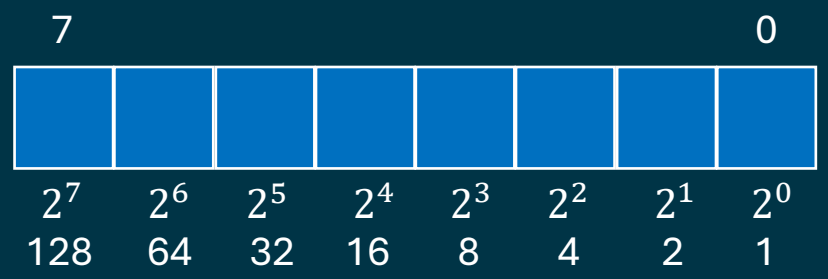
Numbering Systems

- Binary
  - Two digits, designated “0” and “1”
  - Typical prefix for use: `0b` (`b` suffix in MASM)
- Decimal
  - 10 digits (0 to 9), commonly used by humans
  - Typically, no prefix
- Hexadecimal
  - 16 digits (0 to 9, A to F)
  - Compact representation
  - Typical prefix: `0x` (`h` suffix in MASM)
- Octal
  - 8 digits (0 to 7)
  - More compact than binary, not as common as it used to be
  - Typical prefix: `0o`

©2024 Pavel Yosifovich

Number Base Conversions

- Binary to hex
  - Convert every nibble from the LSB to a single hex digit
  - Example: `11`0110`1011=0x36B`
- Binary to decimal
  - Add powers of two for each set bit
  - Example: `10010011=128+16+2+1=147`
- Hex to decimal
  - Add powers of 16 times each hex digit
  - Example: `0x5e3=`$5 * 16^2 + 14 * 16^1 + 3 * 16^0 = 1507$



©2024 Pavel Yosifovich

Two's Complement

- Only needed when interpreting signed integers
- Convenient way to represent negative numbers
- Allow using "normal" addition to get correct results
- If MSB=1, it's a negative number
- To figure out its value
  - Invert every bit, then add 1
  - Alternatively: find the first "1" from the right, then flip all bits to the left of it

©2024 Pavel Yosifovich

Integer Examples

- Byte: `00110001=49` (signed and unsigned)
- Byte: `11111111=255` (unsigned), `-1` (signed)
- Word: `10101010`10101010=0xAAAA=43690` (unsigned)
  - Two's complement: `01010101`01010110=0x5556=21846`
  - Signed value: `-21846`
- Byte: `10000000=128` (unsigned), `-128` (signed)
- Byte: `10000001=129` (unsigned), `-127` (signed)

©2024 Pavel Yosifovich

Sign and Zero Extensions

- Converting a smaller sized number to a larger sized number
  - Needed in certain cases, like performing arithmetic operations
  - What to do with the extra bits?
- Zero extension
  - Fill extra bits with zeros
  - Appropriate for unsigned integers
- Sign extension
  - Fill extra bits with the MSB of the original value
  - Maintains the number's sign

©2024 Pavel Yosifovich

Instruction Operands

- Most instructions require operands (parameters)
- Some require source and destination
- Some have source and/or destination implicit in the instruction
- Operand types
  - Immediate – value is directly used
  - Register – value is provided by a register
  - Memory – value is in memory

©2024 Pavel Yosifovich

Instructions and Operands Examples

Type	Assembly	C/C++ Equivalent
Immediate	`mov rax, 5`	`rax = 5`
	`mul r7, 12`	`r7 *= 12`
	`shl rcx,3`	`rcx <<= 3`
	`xor r8, 2fffh`	`r8 ^= 0x2fff`
Register	`mov rax, rcx`	`rax = rcx`
	`add rax, rdx`	`rax += rdx`
	`mul rbx`	`rdx:rax = rax * rbx`
	`and r5w, 0fff0h`	`r5w &= 0xfff0`
Memory	`mov rax, [r9]`	`rax = *r9`
	`or rcx, [rbx+rsi*4]`	`rcx |= *(rbx + rsi * 4)`
	`mov dword ptr [r6], 11`	`*(unsigned long*)r6 = 11`
	`Shl word ptr [r2], 4`	`*(short*)r2 <<= 4`

©2024 Pavel Yosifovich

Memory Addressing

- Final Address = *`BaseReg + IndexReg*Scale + Disp`*
- `BaseReg` – any general-purpose register
- `IndexReg` – any general-purpose register except RSP
- `Scale` – 0, 1, 2, 4, or 8
- `Disp` – 0, 1, 2, or 4 byte signed offset
- Final address is always 64-bit

©2024 Pavel Yosifovich

Little Endian vs. Big Endian

- The x86 family of processors use Little Endian
  - Multi-byte values stored with LSB in low address and MSB in high address (to be interpreted correctly)
  - Example: 0x1234 at address 0x1000 stored as 0x34 (0x1000) 0x12 (0x1001)
- Big Endian is the opposite

©2024 Pavel Yosifovich

Some Fundamental Instructions

- The MOV instruction
- ADD and SUB instructions
- The CALL and RET instructions
- MASM procedures
- Calling Conventions
- Bitwise Instructions
- Branch Instructions

©2024 Pavel Yosifovich

The MOV Instruction

- `MOV destination, source`
- The most used instruction
- Source may be a register, memory location, or immediate value
- Destination can be a register or memory location
  - Source and destination cannot both be memory locations
- Size of source and destination must match
- The only instruction that allows 64-bit immediate values
- If unclear, must specify explicitly (e.g. `word ptr` in MASM)

©2024 Pavel Yosifovich

Zero and Sign Extension

- Move with zero extension
  - `MOVZX reg16/32/64, reg/mem8/16`
- Move with sign extension
  - `MOVSX reg16/32/64, reg/mem8/16`
- Move with sign extension DWORD
  - `MOVSXD reg64, reg/mem32`

©2024 Pavel Yosifovich

More MOVs

- `MOVBE reg/mem, reg/mem`
  - Move Big Endian
  - Reverses the data while moving
- `MOVD xmm/reg32, reg32/xmm`
- `MOVQ xmm/reg64, reg64/xmm`
  - Move from/to XMM register (lower part)

©2024 Pavel Yosifovich

Simple Arithmetic

- The ADD and SUB instructions
  - `ADD destination, source`
    - destination = destination + source
  - `SUB destination, source`
    - destination = destination - source
- Immediate values are limited to 32-bit
  - If the destination is 64 bit, the immediate value is sign-extended
- ADC (add with carry), SBB (subtract with borrow)

©2024 Pavel Yosifovich

CALL and RET

- `CALL address`
  - Pushes the instruction pointer on the stack
    - Address is typically a procedure name
  - Jumps to the specified address
- `RET [value]`
  - Pops a value (address) from the stack
  - Jumps to that address
  - Optional value is added to the stack pointer after return address is popped
    - Useful for automatic removal of arguments (certain x86 calling conventions)

```
main proc
    call proc1
    ret
main endp

proc1 proc
    ret
proc1 endp
```

©2024 Pavel Yosifovich

Calling External Functions

- Declare function name with `EXTERNDEF` *`name`*`:proc`
- Function can be from an enclosing C application or a Windows API function
- Extern C functions must be marked `extern "C"` if in a C++ compilation unit

```
.code

externdef MessageBoxA:proc

main proc
    sub rsp, 40
    xor rcx, rcx
    mov rdx, offset msg
    mov r8, offset caption
    xor r9, r9
    call MessageBoxA
    add rsp,40
    ret
main endp

.data

caption db "Hello Assembly",0
msg db "This is a cool app!",0

end
```

©2024 Pavel Yosifovich

x64 Calling Convention

- First four integer or pointer arguments passed in `RCX`, `RDX`, `R8`, `R9`
  - For C++, the “this” pointer is passed in RCX
- First four floating point arguments passed in `XMM0-3` (SSE)
- The caller reserves space on the stack for arguments passed in registers
  - Always 32 bytes, even if there are less than 4 arguments
- RAX, RCX, RDX, R8-R11 are volatile
  - RBX, RBP, RSI, RDI, R12-R15 are non-volatile
- If more than 4 arguments - the rest of the arguments are pushed on the stack from right to left

©2024 Pavel Yosifovich

Returning a Value

- A function can return a value
- Technically, any register can be used if it's between cooperating assembly functions
- When working with external code, an agreed upon calling convention must be employed
- The Microsoft x64 calling convention is used by practically all Windows libraries
- The return value must be placed in `RAX` (integer/pointer) or `XMM0` (floating point) registers

©2024 Pavel Yosifovich

x86 Calling conventions

- Win32 (`__stdcall`)
  - Arguments passed on the stack, pushed right to left, callee cleans the stack
- Native C++ ("thiscall")
  - "this" pointer passed in ECX
  - Arguments passed on the stack, pushed right to left, callee cleans the stack
- COM (`__stdcall` for C++ methods)
  - Arguments passed on the stack, pushed right to left
  - "this" pointer pushed on the stack, callee cleans the stack
- `__fastcall`
  - The first 32-bit (or smaller) arguments are passed in ECX and EDX
  - Remaining arguments push on the stack, right to left, callee cleans the stack
- `__cdecl`
  - Arguments pushed right to left, caller cleans the stack
- EAX is the return value (with EDX if 64 bits)

©2024 Pavel Yosifovich

Common Bitwise Instructions

- AND, OR, XOR, NOT logic gates
- Work on a bit-by-bit basis (hence “Bitwise”)
  - `AND destination, source`
    - C/C++: `destination &= source;`
  - `OR destination, source`
    - C/C++: `destination |= source;`
  - `XOR destination, source`
    - C/C++: `destination ^= source;`
  - `NOT destination`
    - C/C++: `destination = ~destination;`

©2024 Pavel Yosifovich

Bitwise Operator Truth Tables

AND truth table

A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1

```
A AND 0 = 0
A AND 1 = A
```

OR truth table

A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1

```
A OR 0 = A
A OR 1 = 1
```

XOR truth table

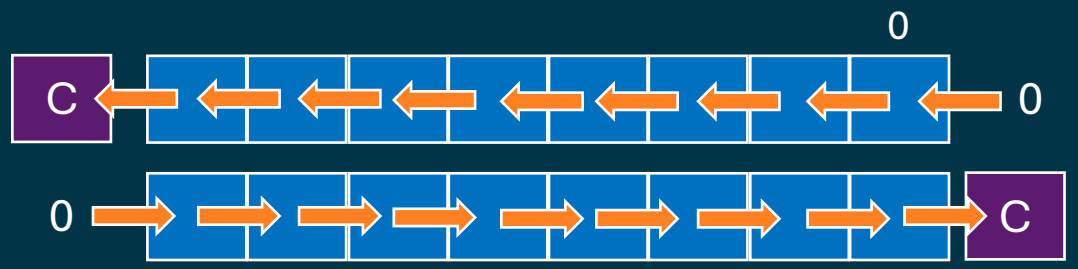
A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

```
A XOR 0 = A
A XOR 1 = NOT A
```

©2024 Pavel Yosifovich

Shifts and Rotates

- The instruction set supports shifting bits (left or right), rotating bits (left or right)
  - `SHL dest, count` (shift left)
  - `SHR dest, count` (shift right)



- `count` can be an immediate value or the CL register
- If `count` is (immediate) 1
  - The dropped bit is placed in the Carry Flag

©2024 Pavel Yosifovich

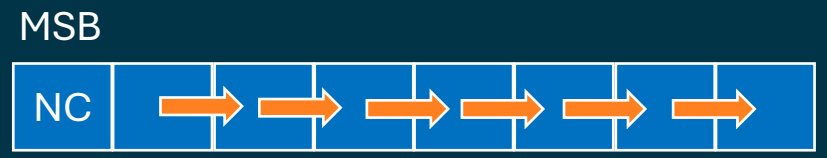
Shift Details

- The Zero flag is set based on the result
- The Overflow flag is set if there is a change in the MSB (only if count is 1), otherwise undefined
- SHL copies the bit shifted in the MSB to the Sign flag
- SHR cleats the Sign flag (zero is always placed in the MSB)
- Shifting left *x* bits is multiplying by 2^x
- Shifting right *x* bits is dividing by 2^x (unsigned)

©2024 Pavel Yosifovich

Arithmetic Shift Right

- Shifting right with SHR divides by powers of two, but only when treating the number as unsigned
- To properly shift right signed numbers, the MSB must not be changed
- This is exactly what the SAR instruction is for
- `SAR dest, count`

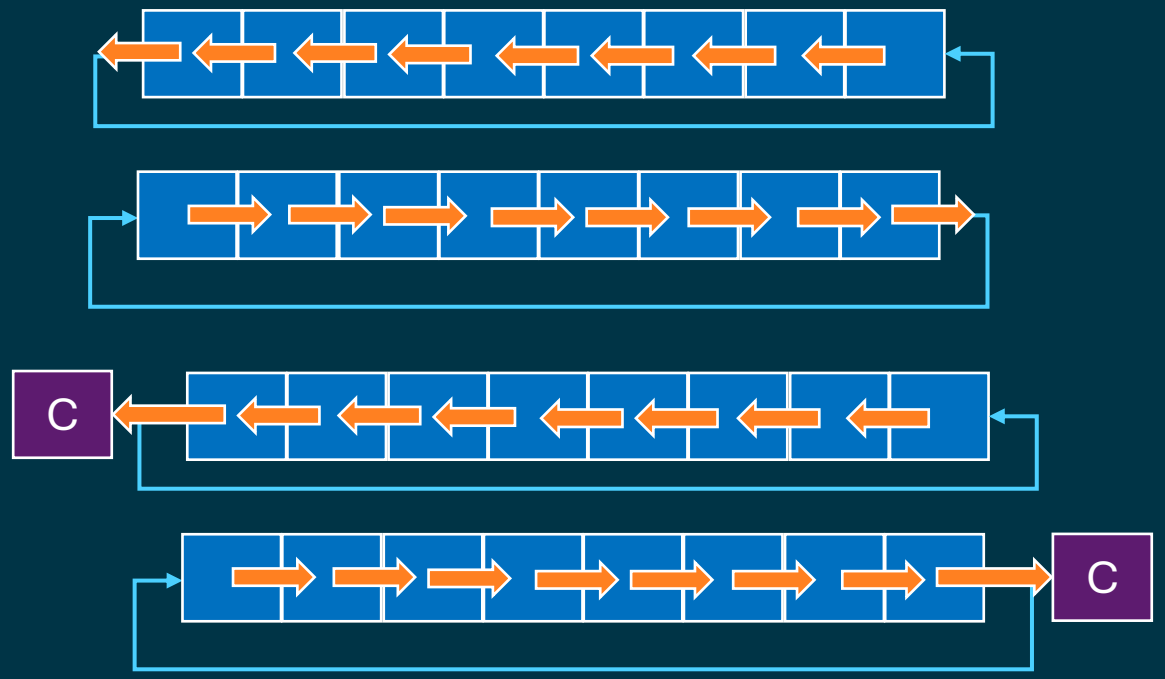


- The Zero and Sign flags are updated
- The Overflow flag is cleared (overflow is not possible)

©2024 Pavel Yosifovich

Rotates

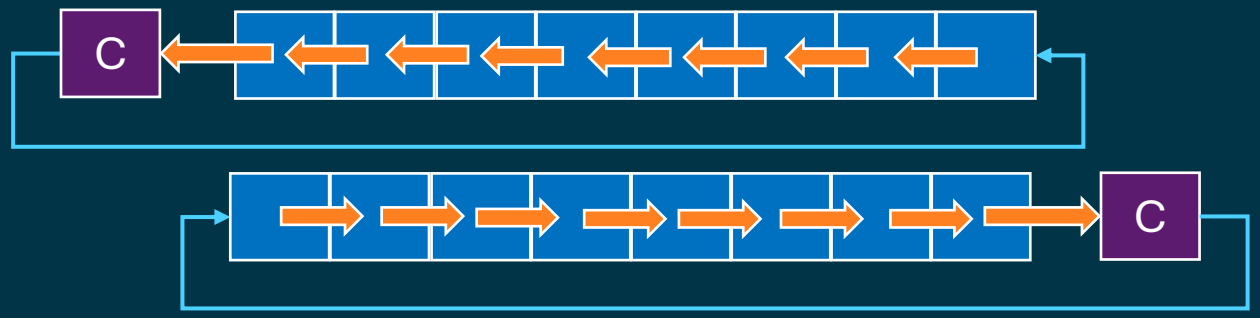
- Similar to shifts, but the shifted-out bit is placed at the other end
  - `ROL dest, count`
  - `ROR dest, count`
- If the rotate count is 1, the shifted-out bit is copied to the Carry flag
- ZF and SF are unaffected
- OF is updated if count=1
  - Left rotate: OF=original MSB XOR (MSB-1)
  - Right rotate: OF=changed MSB XOR (MSB-1)



©2024 Pavel Yosifovich

Rotates with Carry

- It is sometimes useful to involve the Carry in a rotate as an extra bit
  - `RCL dest, count`
  - `RCR dest, count`



- SF and ZF are unaffected
- OF is updated if count=1, undefined otherwise
  - Left rotates: OF is set if the original 2 MSB changed
  - Right rotates: OF=changed MSB XOR (MSB-1)

©2024 Pavel Yosifovich

Floating Point Numbers

- Designed by Intel, later formalized as IEEE 754 standard
- Three parts: Sign, Mantissa, Exponent

Format	Mantissa bits	Exponent bits	Range / precision	Remarks
Single precision (32 bit)	24	8	$\pm 10^{38}$/ ~6 digits	MSB of mantissa is always 1
Double precision (64 bit)	53	11	$\pm 10^{308}$/ ~14 digits	MSB of mantissa is always 1
Extended precision (80 bit)	64	15	$-10^{-4951}\ to + 10^{4932}$/ ~18 digits	Used internally

- Value = $\pm Mantissa * 2^{Exponent}$
- *Exponent* is in excess-*n* format, *Mantissa* is in one's complement

©2024 Pavel Yosifovich

MASM and Floating Point

- MASM supports specifying floating point values with scientific notation just like C/C++/C#/Java, etc.
- Data types: `real4`, `real8`, `real10`
- `real4` is `float` in C, `real8` is `double` in C
- NASM
  - `dd` and `dq` can be used with floating point values

```
; MASM
f1 real4 ?
f2 real4 2.7
pi real4 3.14159
betterpi real8 3.141592653589
d1 real8 ?
d2 real8 1.23456e+10
G real10 ?
```

©2024 Pavel Yosifovich

Jump (Branch) Instructions

- Unconditional jump
  - `JMP target`
- Conditional jumps
  - The `Jcc` family of instructions
  - Use 4 flags to make decisions: Carry (C), Sign (S), Overflow (O), Zero (Z)
- Before a conditional branch, some instruction must be executed to change the flags appropriately

```
xor rcx, rcx
again:
    inc rcx
    jmp again
```

©2024 Pavel Yosifovich

Direct Flag Conditional Jumps

Instruction	Description
`JC target`	Jump if Carry is set
`JNC target`	Jump of Carry is clear
`JO target`	Jump if Overflow is set
`JNO target`	Jump if Overflow is clear
`JS target`	Jump if Sign is set
`JNS target`	Jump if Sign is clear
`JZ target`	Jump if Zero is set
`JNZ target`	Jump if Zero is clear
`JP target`	Jump if Parity is set
`JPE target`	Jump if Parity is clear (even)

- Instructions that cause the flags to update
  - Arithmetic (`ADD`, `SUB`), bitwise (`AND`, `OR`, `XOR`, `NOT`)
- `MOV` instructions do not affect the flags

©2024 Pavel Yosifovich

ADD/SUB and Flags

Flag	ADD	SUB
Carry (C)	Set if unsigned overflow occurs	Set if there is a borrow
Overflow (O)	Set if a signed overflow occurs	Set if there is a signed overflow
Sign (S)	Set if the MSB is set, clear otherwise	Set if the MSB is set, clear otherwise
Zero (Z)	Set if the result is zero, clear otherwise	Set if the result is zero, clear otherwise

```
mov eax, 12
sub eax, 5
sub eax, 9
mov bl, 0x80
add bl, 0x80
sub bl, 4
```

©2024 Pavel Yosifovich

The CMP Instruction

- Same syntax as SUB
- Compares destination and source by subtracting source from destination and setting flags as appropriate
  - The result itself is not stored anywhere
- Using the previously mentioned conditional branch instructions is not as intuitive as we'd like
- Fortunately, other instructions exist for a more intuitive conditional jumps (some are just synonyms for the aforementioned instructions)

©2024 Pavel Yosifovich

Conditional Jump Instructions After CMP

- `CMP left, right`

Instruction	Description	Condition	Jump if…
JE/JZ	Jump if equal/zero	ZF=1	Left equals to right
JNE/JNZ	Jump if not Equal/Zero	ZF=0	Left is not equal to right
JA/JNBE	Jump if above/not below or equal	CF=0 and ZF=0	Unsigned left > unsigned right
JAE/JNB/JNC	Jump if above or equal/not below/carry	CF=0	Unsigned left >= unsigned right
JB/JNAE/JC	Jump if below/not above or equal/carry	CF=1	Unsigned left < unsigned right
JBE/JNA	Jump if below or equal/not above	CF=1 or ZF=1	Unsigned left <= unsigned right
JG/JNLE	Jump of greater/not less or equal	SF=OF and Z=0	Signed left > signed right
JGE/JNL	Jump of greater or equal/not less	SF=OF	Signed left >= signed right
JL/JNGE	Jump if less/not greater or equal	SF!=OF	Signed left < signed right
JLE/JNG	Jump if less or equal/not greater	ZF=1 or SF!=OF	Signed left <= signed right

©2024 Pavel Yosifovich

The TEST Instruction

- `TEST reg/mem, reg/mem/imm`
- Performs an AND operation between the two operands and sets the flags ZF, SF, and PF
  - CF and OF are cleared
  - The result is discarded
- Useful after an operation that changes data but does not change flags
  - E.g., `MOV` instruction
- Example: `TEST EAX, EAX`
  - Sets the flags according to the value in EAX

©2024 Pavel Yosifovich

Summary

- Brief Historical Perspective
- Data Types
- Registers
- Numbering Systems
- Memory Addressing
- Some Basic Instructions

©2024 Pavel Yosifovich

Assembly Programming I

Module 2

©2024 Pavel Yosifovich

Agenda

- MASM and Sections
- Addressing Modes
- The Stack
- Local Variables
- Calling Windows API Functions
- MASM Interlude
- Summary

©2024 Pavel Yosifovich

MASM and Sections

- MASM supports the following sections
  - `.code` (execute / read)
  - `.data` (read / write)
  - `.data?` (read / write), uninitialized
  - `.const` (read only)
- Can appear multiple times
  - Will be consolidated by MASM
- Mapped as PE sections

```
.code
main proc
    ret
main endp

.data
db 1,2,3,4
dw 1000h, 2000h, 3000h, 4000h

.data?
dd 10 dup (?)
myvalue db ?

.const
Count dd 12345678h
pi real4 3.1415926
C dq 299792458
```

©2024 Pavel Yosifovich

The LEA Instruction

- Load Effective Address
- `LEA register, mem_var`
- Places the address *mem_var* points to in the register
  - Similar to "address of" C operator
- Basic functionality achievable with `MOV` as well
  - Use the `offset` prefix in MASM
- More flexible than `MOV` for the same purpose
  - E.g. `LEA RAX, [RBX + 8]` vs. `MOV RAX, RBX + 8` (illegal)
- Example
  - `MOV RAX, [1000]` vs. `LEA RAX, [1000]`

©2024 Pavel Yosifovich

Addressing Modes

- Ways to specify access to memory
  - Register
  - RIP relative
  - Register indirect
  - Indirect + offset
  - Scaled index

©2024 Pavel Yosifovich

Register Addressing Mode

- Use of registers is fastest and produces more compact code
- Size of registers must match

```
mov rax, rcx                ; 48 89 C8
mov ax, bx                  ; 66 89 D8
add ebx, esi                ; 01 F3
mov ah, al                  ; 88 C4
mov bx, bx                  ; 66 89 DB
```

©2024 Pavel Yosifovich

RIP (PC) Relative Addressing Mode

- The most common when accessing memory
- Memory referenced is relative to the address of the next instruction
  - Sometimes referred to as *Position Independent Code* (PIC)
- Relative offset is limited to a signed 32 bit
  - Meaning access is limited to the range -2GB to +2GB
- MASM uses it by default if possible
  - With NASM, use `default rel` directive
- The LEA instruction is helpful

©2024 Pavel Yosifovich

Example: Relative vs. Absolute

- Using NASM

```
default rel
lea rax, mydata

default abs
lea rax, mydata

hlt

mydata db "hello!"
```

```
0x00001000 lea   rax, [rip + 9]  ; 48 8D 05 09 00 00 00
0x00001007 lea   rax, [0x1010]   ; 48 8D 04 25 10 10 00 00
0x0000100F hlt                   ; F4
0x00001010 push  0x6f6c6c65      ; 68 65 6C 6C 6F
```

©2024 Pavel Yosifovich

Register Indirect Addressing Mode

- Access memory address stored in a register
- Indirect register can be any general purpose 64-bit register
- Examples
  - `mov [rax], ebx`
  - `mov [rcx], al`
  - `mov [rbx], dword ptr 12`

©2024 Pavel Yosifovich

Indirect + Offset Addressing Mode

- Adds a signed 32-bit constant to a register before accessing the memory
- Examples
  - `mov [rax + 10h], cx`
  - `mov al, [rsi - 4]`

©2024 Pavel Yosifovich

Scaled Indexed Addressing Modes

- Combines two registers and an optional displacement
- `[Base + Index*Scale + Displacement]`
- Base is any 64-bit general purpose register
- Index is any 64-bit general purpose register except RSP
- Scale can be 1,2,4 or 8
- Displacement is a signed 32-bit value
- Examples
  - `mov [rax + rsi * 2], edx`
  - `add rbx, [rsi + rdi * 8 + 72]`

©2024 Pavel Yosifovich

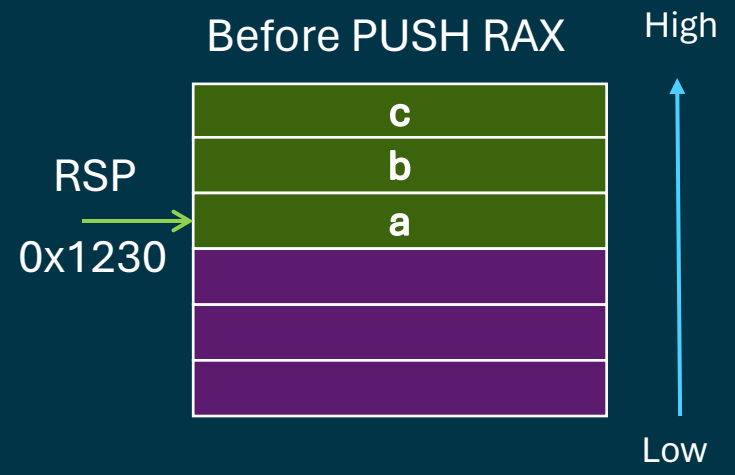
The Stack

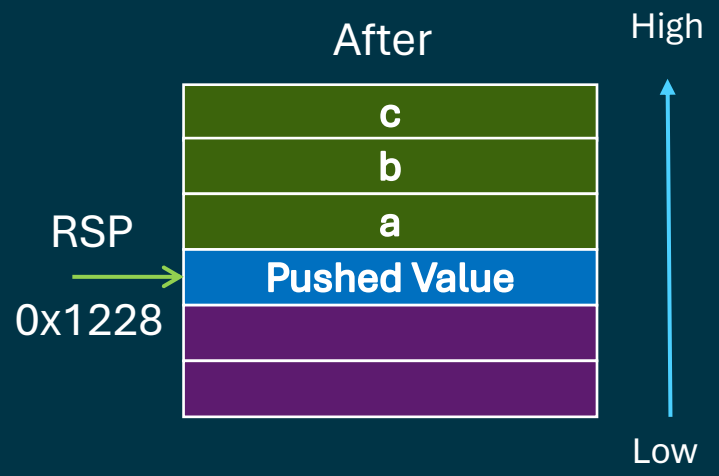
- The stack is a memory block that is managed as a Last-In-First-Out (LIFO) data structure
  - On x86 architectures, the stack grows downwards in memory
- The current stack pointer is managed by the RSP register
  - Points to the top of the stack where the last valid value is
- Pushing a value
  - Decrements RSP by the size of the operand
  - Copies the value to the memory pointed to by RSP
- Popping a value into a register
  - Copies the top-of-stack value to the register
  - Increments RSP by the size of the operand
- CALL and RET instructions always use the stack

©2024 Pavel Yosifovich

The PUSH Instruction

- The PUSH instruction pushes a value onto the stack
  - `PUSH register` (16 bit or 64 bit)
  - `PUSH memory` (16 bit or 64 bit)
  - `PUSH constant` (16 bit or 32 bit)
    - 32-bit constant is sign-extended to 64-bit
- Best to avoid 16-bit PUSH
- Keep stack 8-byte aligned





©2024 Pavel Yosifovich

The POP Instruction

- Get data item off the stack
  - `POP register` (16 bit or 64 bit)
  - `POP memory` (16 bit or 64 bit)
- Operation
  - Value from the `[RSP]` is copied to the operand
  - RSP incremented by operand size (2 bytes or 8 bytes)
- The popped value is still in memory

©2024 Pavel Yosifovich

Manipulating the Stack

- The stack is just memory, like any other
- Although the PUSH/POP instructions manipulate the stack, direct manipulation is possible
  - Working with the RSP register directly
- Examples
  - `SUB RSP, 16` (make room for 16 bytes on the stack)
  - `ADD RSP, 16` (discard 16 bytes from top of stack)
  - `MOV RAX, [RSP]` (get top of stack value)
  - `MOV RBX, [RSP+8]`

©2024 Pavel Yosifovich

Invoking Windows APIs

- First four arguments passed in registers
- Rest of arguments passed on the stack
  - Pushed right to left
    - Using PUSH instructions or direct stack manipulation
  - "Spill" area (32 bytes) must be reserved on the stack
  - Stack must be 16-bytes aligned before a `CALL` instruction
- MASM
  - `importlib` *`libraryname`*
  - *`function`* `proto` or `externdef` *`function`*`:proc`

©2024 Pavel Yosifovich

Invoking Windows APIs Example 1

```
externdef CreateFileA:proc
externdef WriteFile:proc
externdef CloseHandle:proc
externdef GetLastError:proc

OPEN_ALWAYS = 4
GENERIC_WRITE = 40000000h

main proc
    lea rcx, filename        ; filename
    mov edx, GENERIC_WRITE   ; desired access
    xor r8d, r8d             ; share mode
    xor r9, r9               ; security attributes
    push 0                   ; template file
    push 0                   ; flags and attributes
    push OPEN_ALWAYS         ; creation disposition
    sub rsp, 20h
    call CreateFileA
    test eax, eax
    js error
```

```
    mov rcx, rax
    lea rdx, text
    mov r8, sizeof text
    lea r9, written
    mov rbx, rcx
    push 0
    and rsp, -16
    call WriteFile
    test eax, eax
    jz error
    mov rcx, rbx
    call CloseHandle
    test eax, eax
    jz error
    add rsp, 20h + 28h
    ret
error:
    call GetLastError
    ret
main endp

.data
filename db "C:\temp\test.txt",0
text db "some nice text in file"
written dd ?
```

©2024 Pavel Yosifovich

Managing the Stack

- It's critical to revert the stack pointer to the same value it had when entering a function / procedure
  - Can be done manually with arithmetic operations on RSP / PUSH / POP, but error prone
  - Typical solution is to save RSP in RBP at the beginning of the function and restore it before exiting the function
  - The old RBP should be preserved

```
; prolog
push rbp
mov rbp, rsp
```

```
; epilog
mov rsp, rbp
pop rbp
```

©2024 Pavel Yosifovich

Local Variables

- Local variables are “allocated” on the stack
  - Just subtracting from the stack pointer the total size of local variables
- The ENTER instruction
  - `ENTER n, 0`
  - Saves old RPB on the stack, copies RSP to RBP, and finally subtracts the instructed number of bytes (*n*) from RSP for the local variables
- The LEAVE instruction
  - Copies RBP back to RSP
  - Pops the old RBP back

©2024 Pavel Yosifovich

Invoking Windows APIs Example 2

```
externdef CreateFileA:proc
externdef WriteFile:proc
externdef CloseHandle:proc
externdef GetLastError:proc

OPEN_ALWAYS = 4
GENERIC_WRITE = 40000000h

main proc
    push rbp
    mov rbp, rsp
    sub rsp, 16
    written equ [rbp - 4]
    lea rcx, filename        ; filename
    mov edx, GENERIC_WRITE   ; desired access
    xor r8d, r8d             ; share mode
    xor r9, r9               ; security attributes
    push 0                   ; template file
    push 0                   ; flags and attributes
    push OPEN_ALWAYS         ; creation disposition
    sub rsp, 20h
    call CreateFileA
    test eax, eax
    js error
```

```
    mov rcx, rax
    lea rdx, text
    mov r8, sizeof text
    lea r9, written
    mov rbx, rcx
    and rsp, -16
    push 0
    sub rsp, 8
    call WriteFile
    test eax, eax
    jz error
    mov rcx, rbx
    call CloseHandle
    test eax, eax
    jz error
    ret
error:
    call GetLastError
    ret
main endp

.data
filename db "C:\temp\test.txt",0
text db "some nice text in file"
```

©2024 Pavel Yosifovich

The LOCAL Directive

- Can be placed as the first thing in a PROC
- `LOCAL var1:type, var2:type, …`
- MASM inserts a prolog with the calculated size of the local variables
  - Also makes sure the stack is 16 bytes aligned
  - Assumes it's address ends with 8 hex upon entry to the function
- Any RET instruction causes an additional LEAVE instruction to be inserted before the RET

©2024 Pavel Yosifovich

Invoking Windows APIs Example 3

```
externdef CreateFileA:proc
externdef WriteFile:proc
externdef CloseHandle:proc
externdef GetLastError:proc

OPEN_ALWAYS = 4
GENERIC_WRITE = 40000000h

main proc
    local written
    lea rcx, filename        ; filename
    mov edx, GENERIC_WRITE   ; desired access
    xor r8d, r8d             ; share mode
    xor r9, r9               ; security attributes
    push 0                   ; template file
    push 0                   ; flags and attributes
    push OPEN_ALWAYS         ; creation disposition
    sub rsp, 28h
    call CreateFileA
    test eax, eax
    js error
```

```
    mov rcx, rax
    lea rdx, text
    mov r8, sizeof text
    lea r9, written
    mov rbx, rcx
    push 0
    and rsp, -16
    sub rsp, 20h
    call WriteFile
    test eax, eax
    jz error
    mov rcx, rbx
    call CloseHandle
    test eax, eax
    jz error
    ret
error:
    call GetLastError
    ret
main endp

.data
filename db "C:\temp\test.txt",0
text db "some nice text in file"
```

©2024 Pavel Yosifovich

The INVOKE Macro

- Calling Windows APIs manually is tedious and error prone
- In 32-bit mode, MASM supports the INVOKE macro
  - Makes it easy to invoke functions
  - Example

```
invoke MessageBoxA, 0, addr msg, addr caption, 0
```

- This macro is not supported in 64-bit mode
- However, the MASM64 SDK project has an implementation of this macro for 64-bit

©2024 Pavel Yosifovich

Using the MASM64 SDK

- Clone or download the repo
  - https://github.com/PawelWorzala666/masm64-SDK
- The SDK assumes it's installed in the root directory of a drive
  - If you put in in a different location, change the include file paths and / or add additional paths in the project's properties
- To include everything, add

```
include \masm64\include64\masm64rt.inc
```

- Write code!

©2024 Pavel Yosifovich

Saving / Restoring Flags

- The Flags register can be saved/restored to/from the stack
  - PUSHF (push 16-bit FLAGS)
  - PUSHFQ (push 64-bit RFLAGS)
  - POPF (pop 16-bit value into FLAGS)
  - POPFQ (pop 64-bit value into RFLAGS)

©2024 Pavel Yosifovich

INC and DEC

- The `INC` instruction increments its operand by one
  - `INC register/memory`
- The `DEC` instruction decrements its operand by one
  - `DEC register/memory`
- Slightly more efficient than using ADD/SUB with 1
- However, the Carry flag is unaffected by `INC`/`DEC`

©2024 Pavel Yosifovich

MASM Interlude

- MASM Options
- Typedef
- Constant Expressions
- Type Coercion
- Character Strings
- Arrays
- Structs and Unions

©2024 Pavel Yosifovich

MASM Options

- The `OPTION` directive
  - Various options affecting MASM behavior
- Examples
  - `OPTION casemap:none` (make MASM case sensitive)
    - Default is `NOTPUBLIC`
  - `OPTION DOTNAME` (enable using dot as first character of an identifier)
    - `OPTION NODOTNAME`
  - `OPTION SCOPED` (labels are local to procedures – default)
    - `OPTION NOSCOPED`

©2024 Pavel Yosifovich

Typedef

- MASM supports a typedef directive, similar to C/C++
- *newName* `typedef` *existingName*
- Examples

```
integer typedef sdword
length typedef word
float typedef real4
```

©2024 Pavel Yosifovich

Constant Expressions

- *identifier = constantExpression*
- *identifier* EQU *constantExpression*
- *constantExpression* must be assembly time constant
  - Can include calculation based on previous constant declarations and arithmetic operators and functions
  - MASM will use the result wherever it appears in code
- With =, the identifier's value can be changed later
  - Applies from the point of change
  - Constants defined with = must be representable as a 64-bit integer or smaller

©2024 Pavel Yosifovich

The TEXTEQU Directive

- Similar to a `#define` preprocessor directive in C/C++
  - Does text replacement
  - Place text in angle brackets
- *identifier* TEXTEQU <*text*>
- Example
  - `copy textequ <mov>`
  - `copy EAX, 5`

©2024 Pavel Yosifovich

$ and THIS

- Return the offset in the current section
- Examples
  - `JMP $`
  - `label1 equ $`
  - `someData db 1, 2, 3, 4, 5, 6`
  - `sizeOfData = $ - someData`
- $ has a default type of statement label
- THIS can be typed
  - `label1 label byte`
  - `label1 equ this byte`

©2024 Pavel Yosifovich

Type Coercion

- Some instructions will not compile as they are ambiguous or the wrong size
- Examples
  - `INC [RAX]`
  - `MyData byte 10 dup(?)`
  - `MOV eax, MyData`
- Type coercion is required
  - `INC word ptr [RAX]`
  - `MOV eax, dword ptr MyData`

©2024 Pavel Yosifovich

Character Strings

- MASM allows using `db` or `byte` to write literal ASCII strings surrounded by quotation marks
  - Can add integer literals if needed
- Example: `Hello db “Hello, world!”, 13, 10, 0`
- String related functions are too high level to be provided by machine instructions
  - The so-called “string” instructions are really about arrays
- Can invoke CRT functions (from <*string.h*>)

©2024 Pavel Yosifovich

Arrays

- Arrays are just contiguous data items in memory
- MASM provides the DUP directive to make it easier to allocate static arrays

```
arrayName type n DUP (value)
```

- Examples

```
array1 byte 256 dup (?)
array2 dd 20 dup (0)
array3 real4 -1.0, 2.0, 0.0, 3.1, 125.0
```

©2024 Pavel Yosifovich

Structures

- The `struct` MASM keyword can be used to define structures similarly to C/C++

```
Point struct
    x dd ?
    y dd ?
Point ends

Person struct
    name byte 30 dup (?)
    age sbyte ?
    weight real4 ?
Person ends
```

```
.data
point1 Point <10, 20>
point2 Point { 4, 5 }
Bart Person <"Bart Simpson", 10, 50.0>
Homer Person { "Homer Simpson" , 40, 100.0 }
pointfmt db "(%d, %d)", 0
```

```
lea rax, point1
lea rcx, pointfmt
mov edx, [rax].Point.y
mov r8d, [rax].Point.x
call printf
```

- Uninitialized structure: `John Person {}` or `<>`

©2024 Pavel Yosifovich

Structures and Alignment

- Structure members are contiguous in memory
  - No padding for alignment done by default
- The `align` directive allows specifying the required alignment (going forward)
- Adding an alignment value to the struct itself (1, 2, 4, 8, or 16) tells MASM to align members on their size and the specified alignment, which ever is smaller

```
Test1 struct
    b1 byte ?
    align 4
    d1 dword ?
    f1 real8 ?
Test1 ends
```

```
Test1 struct 4
    b1 byte ?
    d1 dword ?
    f1 real8 ?
Test1 ends
```

©2024 Pavel Yosifovich

More on Structures

- The `SIZEOF` operator can be used to get the size of a structure
  - Works on other types as well
- Structures can be nested to any depth
- Arrays within structures are initialized with nested `{}`
- Arrays of structures can be created as well

©2024 Pavel Yosifovich

Unions

- Just like structures, unions can be defined
- `unionName union`
  `declarations`
  `unionName ends`
- *Declarations* are syntactically identical to structures
  - Yes, the declaration ends with `ends`
- All union members start in the same address
- Unions and structures can be combined (nested) to any depth

©2024 Pavel Yosifovich

Summary

- MASM and Sections
- Addressing Modes
- The Stack
- Local Variables
- Calling Windows API Functions
- MASM Interlude

©2024 Pavel Yosifovich

Assembly Programming II

Module 3

©2024 Pavel Yosifovich

Agenda

- Procedures
- Stack Frames
- Multiplication and Division
- String Instructions
- Bit Manipulation
- Summary

©2024 Pavel Yosifovich

Procedures

- Defining Procedures

```
procName PROC [options] [parameters]
   [instructions]
procName ENDP
```

- Options include procedure visibility
  - PUBLIC (visible to other modules - default)
  - PRIVATE (private to this module)
  - EXPORT (public and exported, useful if building a DLL)
- Default visibility can be changed with OPTION PROC:*visibility*

©2024 Pavel Yosifovich

Labels in Procedures

- Labels in procedures are local by default
  - Not visible to other procedures
- A global label can be defined by appending two colon characters (instead of one)
  - Jump instructions can use it from outside the procedure
  - Usually not a good idea

©2024 Pavel Yosifovich

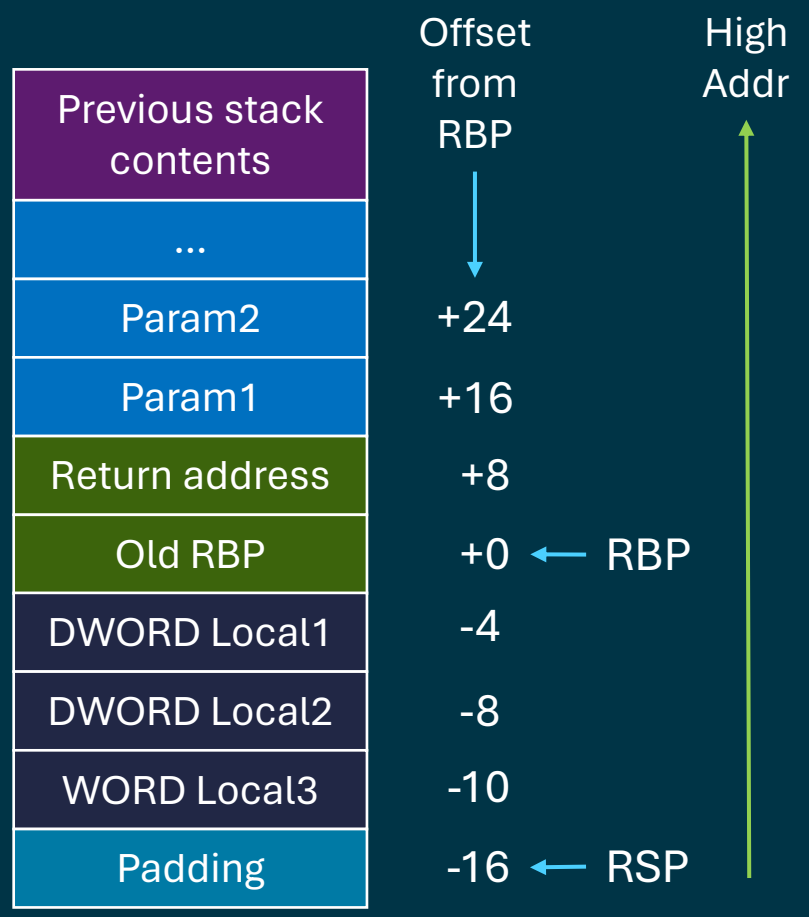
Saving Registers State

- A procedure is likely to use registers
- Callers might expect registers to maintain their values across calls
- It's considered a good practice to preserve the original values of changed registers
  - PUSH/POP can be used
- Preserving certain registers is mandatory if using the Windows x64 calling convention
  - RBX, RBP, RSI, RDI, R12-R15 must be preserved

©2024 Pavel Yosifovich

Stack Frames

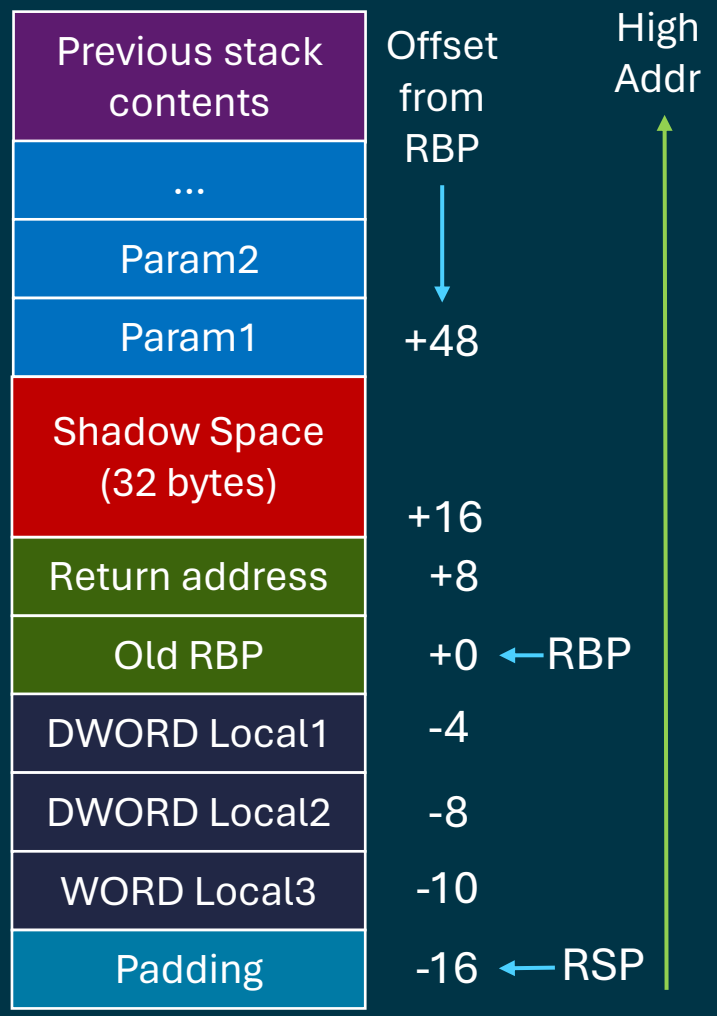
- Sometimes called *Activation Records*
- Before calling a procedure
  - The caller places arguments on the stack
- Caller issues the CALL instruction
  - Pushes RIP onto the stack
- The callee (procedure)
  - Typically has prolog code
  - May allocate local variables



©2024 Pavel Yosifovich

x64 Windows Stack Frames

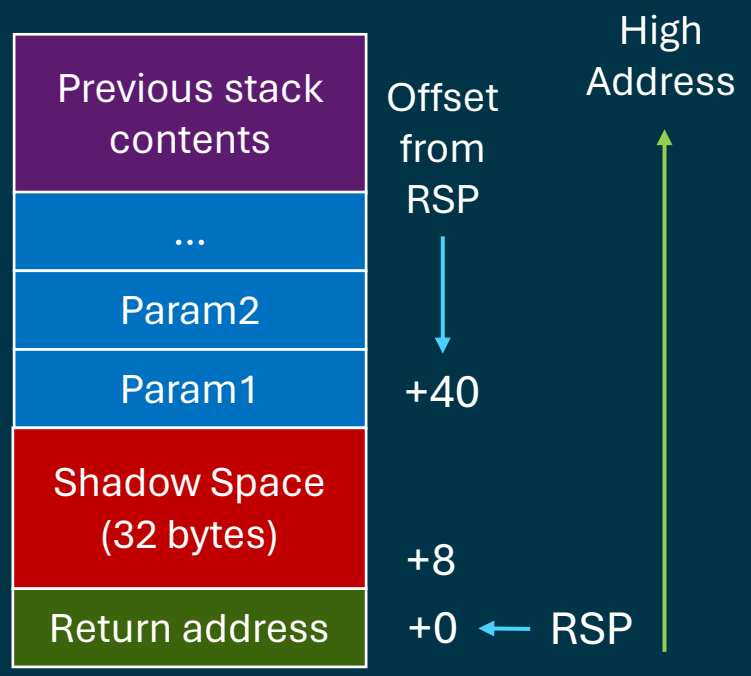
- The Windows x64 calling convention requires a “shadow space” for 4 arguments (32 bytes)
  - Even if the function accepts less than 4 parameters
- Parameters are pushed from right to left so they appear “in order” in memory



©2024 Pavel Yosifovich

Simple x64 Windows Stack Frames

- No local variables
- No prolog code
- "Shadow Space"
- Parameters accessed from RSP+40 onwards



©2024 Pavel Yosifovich

Example: Simple Stack Frame

```c
extern "C" int Sum(int a, int b, int c, int d, int e, int f, int g, int h);

int main() {
    int a = 1, b = 12, c = -3, d = 4;
    int e = 5, f = -6, g = 7, h = 8;
    int sum = Sum(a, b, c, d, e, f, g, h);
    printf("Sum of %d, %d, %d, %d, %d, %d, %d, %d is %d\n",
        a, b, c, d, e, f, g, h, sum);

    return 0;
}
```

```asm
.code

Sum proc
    add ecx, edx ; a + b
    add r8d, r9d ; c + d
    add ecx, r8d

    add ecx, [rsp+40] ; e
    add ecx, [rsp+48] ; f
    add ecx, [rsp+56] ; g
    add ecx, [rsp+64] ; h
    mov eax, ecx
    ret
Sum endp

end
```

©2024 Pavel Yosifovich

Exercise: Construct Stack Frame

- Disassembly of `User32!CreateWindowExW`
- No requirement to use RBP

```
HWND CreateWindowExW(
  DWORD     dwExStyle,
  LPCWSTR   lpClassName,
  LPCWSTR   lpWindowName,
  DWORD     dwStyle,
  int       X,
  int       Y,
  int       nWidth,
  int       nHeight,
  HWND      hWndParent,
  HMENU     hMenu,
  HINSTANCE hInstance,
  LPVOID    lpParam);
```

```
mov   r11,rsp
sub   rsp,88h
xor   eax,eax
mov   word ptr [rsp+78h],ax
mov   dword ptr [rsp+70h],eax
mov   dword ptr [rsp+68h],40000000h
mov   dword ptr [rsp+60h],eax
mov   rax,qword ptr [rsp+0E8h]
mov   qword ptr [r11-30h],rax
mov   rax,qword ptr [rsp+0E0h]
mov   qword ptr [r11-38h],rax
mov   rax,qword ptr [rsp+0D8h]
mov   qword ptr [r11-40h],rax
mov   rax,qword ptr [rsp+0D0h]
mov   qword ptr [r11-48h],rax
mov   eax,dword ptr [rsp+0C8h]
mov   dword ptr [rsp+38h],eax
mov   eax,dword ptr [rsp+0C0h]
mov   dword ptr [rsp+30h],eax
mov   eax,dword ptr [rsp+0B8h]
mov   dword ptr [rsp+28h],eax
mov   eax,dword ptr [rsp+0B0h]
mov   dword ptr [rsp+20h],eax
call  USER32!CreateWindowInternal
add   rsp,88h
ret
```

©2024 Pavel Yosifovich

Procedure Prolog and Epilog

- Prolog
  - ```
    PUSH RBP
    MOV RBP, RSP
    SUB RSP, size_of_local_vars_plus_padding
    ```
- Stack pointer is 16-byte aligned
- Epilog
  - ```
    MOV RSP, RBP
    POP RBP
    RET
    ```
- Caller is responsible for cleaning up arguments pushed on the stack
  - Allows implementing functions like `printf` (variable number of arguments)

©2024 Pavel Yosifovich

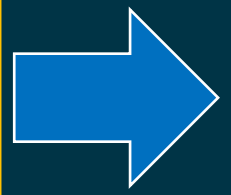
Parameter Allocation and Cleanup

- Callers need to allocate “shadow space” and potentially parameters on the stack
  - Cleanup afterwards
  - Stack pointer must be 16-byte aligned prior to CALL instruction
- With multiple calls, multiple ADD/SUB RSP may be needed
- Optimization
  - Use a single ADD/SUB with the maximum required size for parameters and shadow space

©2024 Pavel Yosifovich

Parameter Allocation Example

```
SUB RSP, param1_size
CALL proc1
ADD RSP, param1_size

SUB RSP, param2_size
CALL proc2
ADD RSP, param2_size
```



```
SUB RSP, max_param_size
CALL proc1

CALL proc2
ADD RSP, max_param_size
```

©2024 Pavel Yosifovich

Passing Floating Point Arguments

- The first 4 parameters in the Windows x64 calling convention are passed in registers
  - RCX, RDX, R8, R9 for integer/pointer values
  - XMM0, XMM1, XMM2, XMM3 for floating point values
- Setting values for XMM registers
  - MOVSD XMM*n*, XMM*m*/*memory* (double precision)
  - MOVSS XMM*n*, XMM*m*/*memory* (single precision)

©2024 Pavel Yosifovich

Declaring Parameters in PROC

- PROC declarations can include parameters
  - *procName* PROC *param1Name:type,…*
- MASM adds a standard prolog
- Accessing the parameters "normally" uses RBP offsets under the covers
  - Does not consider the calling convention
  - First four parameters use the "spill" space (RPB+16, RBP+24, …)
  - Fifth parameter is at RBP+40, sixth at RBP+48, etc.

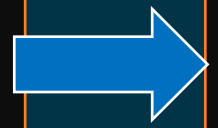
```
main proc
    push 1
    push 2
    push 3
    call calc
    ret
main endp

calc proc a:dword, b:dword, c:dword
    mov eax, a
    add eax, b
    sub eax, c
    ret
calc endp
```

©2024 Pavel Yosifovich

Indirect Calls

- The CALL instruction supports indirect calls
  - `CALL address`
  - `CALL reg(RIP=reg)`
  - `CALL [reg](RIP=*(QWORD*)reg)`
  - `CALL [mem](RIP=*(QWORD*)mem)`

```
call 0x1234
call rax
call [rax]
call [0x1234]
call [rax + 8*rbx + 48]
```



```
0x00001000 call     0x1234                    ; E8 2F 02 00 00
0x00001005 call     rax                       ; FF D0
0x00001007 call     qword ptr [rax]           ; FF 10
0x00001009 call     qword ptr [0x1234]        ; FF 14 25 34 12 00 00
0x00001010 call     qword ptr [rax + rbx*8 + 0x30] ; FF 54 D8 30
```

©2024 Pavel Yosifovich

Unconditional Jumps

- The JMP instruction supports indirect as well as direct jumps
  - `JMP` *`address`*
  - `JMP` *`register`*
  - `JMP` *`memory`*
- Examples of indirect jumps
  - `JMP RBX`
  - `JMP [RBX]`
  - `JMP [RBX + 100]`
  - `JMP [RBX + RSI*4 + 32]`

©2024 Pavel Yosifovich

Conditional Jumps

- Conditional jumps use a relative offset
  - 1 byte (-128 to +127)
  - 4 byte (-2GB to +2GB)
- The assembler will choose the smaller of the two
- Trampolines
  - Needed in a case where a 4-byte jump is insufficient

```
  Jcc skip        ; use the opposite condition
  mov r8, destination    ; 64 bit allowed
  jmp r8
skip: …
```

©2024 Pavel Yosifovich

Conditional Move Instructions

- Allow moving a value if a condition is satisfied
  - `CMOVcc register, register/memory`
- Destination must be a register
- Supported registers are 16, 32, and 64 bit

©2024 Pavel Yosifovich

Loops

- Typical loop uses registers as counters
  - Counting down if possible (e.g., `DEC` instruction)
  - Avoids the need for a `CMP` instruction
- The `LOOP` instruction
  - Decrement RCX and if not zero, jumps to the address (8 bit relative)
  - `LOOP address`
- `LOOPZ/E`, `LOOPNZ/NE` instructions
  - Same as loop but add a condition for ZF
  - LOOPZ/E: decrement RCX, jump if RCX is non-zero and ZF=1
  - LOOPNZ/NE: decrement RCX, jump if RCX is non-zero and ZF=0

©2024 Pavel Yosifovich

Unsigned Multiplication

- `MUL reg/mem`
- Multiplies AL/AX/EAX/RAX by the given operand and stores the result in AX, DX:AX, EDX:EAX, or RDX:RAX
  - Immediate values not supported
- If the upper half of the product is non-zero, CF and OF are set, otherwise cleared
  - Other flags are undefined
- Examples

```
mov eax, 12
mov ebx, 5
mul ebx
```

©2024 Pavel Yosifovich

Signed (Integer) Multiplication

- `IMUL` *`reg/mem`*
  - Multiply AL/AX/EAX/RAX by the operand and stores result in AX, DX:AX, EDX:EAX, or RDX:RAX
- `IMUL` *`destreg, reg/mem`*
  - Multiply *destreg* by the second operand and stores result back in *destreg*
- `IMUL` *`destreg, reg/mem, imm`*
  - Multiply second operand by *imm* and store in *destreg*

©2024 Pavel Yosifovich

Unsigned Division

- `DIV reg/mem`

Division size	Dividend	Divisor	Quotient	Remainder	Quotient range
Word/byte	AX	reg/mem8	AL	AH	`-128 to +127`
Dword/word	DX:AX	reg/mem16	AX	DX	`-32768 to +32767`
Qword/dword	EDX:EAX	reg/mem32	EAX	EDX	$-2^{31}\ to + 2^{31} - 1$
Oword/qword	RDX:RAX	reg/mem64	RAX	RDX	$-2^{63}\ to + 2^{63} - 1$

- Overflow generates a #DE exception (rather than setting OF)
- Dividing by zero generates divide-by-zero exception

©2024 Pavel Yosifovich

Signed Division

- `IDIV reg/mem`

Division size	Dividend	Divisor	Quotient	Remainder	Quotient range
Word/byte	AX	reg/mem8	AL	AH	255
Dword/word	DX:AX	reg/mem16	AX	DX	65535
Qword/dword	EDX:EAX	reg/mem32	EAX	EDX	$2^{32} - 1$
Oword/qword	RDX:RAX	reg/mem64	RAX	RDX	$2^{64} - 1$

- The sign extension instructions can be used to avoid overflow (`CBW`, `CWD`, `CDQ`, `CQO`)

©2024 Pavel Yosifovich

String Instructions

- Work on array of 1, 2, 4, or 8 bytes
- String instructions: MOVS*x*, CMPS*x*, LODS*x*, STOS*x*
  - Where "x" can be B, W, D, or Q
- The instructions work with predefined registers
  - RSI (source index), RDI (destination index), RCX (count)
  - AL, AX, EAX, RAX
  - The Direction flag
- Require a "Repeat" prefix to work on more than one item

©2024 Pavel Yosifovich

The “Repeat” Prefixes

- Specified before the string instruction itself
  - If not specified, string instruction works on the first (current) item only
- REP
  - Repeats the operation RCX times
- REPZ / REPE
  - Repeat the operation RCX times while ZF=1 (stop when either fails)
- REPNZ / REPNE
  - Repeat the operation RCX times while ZF=0 (stop when either fails)

©2024 Pavel Yosifovich

The Direction Flag

- Controls the “direction” strings are processed
- After every string item operation, RSI and RDI are incremented or decremented based on the Direction flag
  - If clear, RSI and RDI are incremented by the number of bytes of the item size (1, 2, 4, or 8)
  - If set, RSI and RDI are decremented by the number of bytes of the item size (1, 2, 4, or 8)
- CLD clears the Direction flag, STD sets it
- For x64 Windows ABI, DF must be clear before a CALL

©2024 Pavel Yosifovich

MOVS Example

```
.code
cld
lea rsi, buffer1
lea rdi, buffer2
mov rcx, bufferSize
rep movsb
hlt

.data
buffer1 db "hello, world!",13,10,0
bufferSize EQU $-buffer1
buffer2 db bufferSize dup(0)
```

- What would happen if MOVSB is replaced with MOVSW?
- If buffers overlap, move in reverse direction from highest address to lowest address
- Generally, using MOVSQ is fastest

©2024 Pavel Yosifovich

String Instructions Description

- STOS*x*
  - Store the value in AL/AX/EAX/RAX in RDI
  - Typical usage is to initialize an array to zeros
- LODS*x*
  - Copies the value pointed to by RSI to AL/AX/EAX/RAX
  - REP prefix makes little sense
- CMPS*x*
  - Compare two arrays, optionally until a match is found/not found
- SCAS*x*
  - Scans an array pointed by RDI, searching for the value in AL/AX/EAX/RAX

©2024 Pavel Yosifovich

BIT Manipulation

- The most common operations for bit manipulation consist of the bitwise operators and shifts/rotates
  - `AND`, `OR`, `XOR`, `NOT`, `SHL`, `SHR`, `SAR`, `ROL`, `ROR`, `RCR`, `RCL`
- Other instructions exist
  - Bit test
    - `BT reg/mem, reg/mem/imm`
    - Copies the bit number specified in the second operand of the first operand to the Carry flag
    - Example: `BT EAX, 5` (copy bit 5 of EAX to CF)

©2024 Pavel Yosifovich

More Bit Tests

- `BTS reg/mem, reg/mem/imm` (Bit test and set)
- `BTR reg/mem, reg/mem/imm` (Bit test and reset)
- `BTC reg/mem, reg/mem/imm` (Bit test and complement)
- Same as `BT`, but the bit in question is set/reset/complemented after being copied to CF
- In many cases, `BTS/R/C` are used without the “test” part – just to set / reset / complement the required bit
  - Simply ignoring the Carry bit

©2024 Pavel Yosifovich

Bit Scan (Forward / Reverse)

- `BSF` *`destreg`*`, reg/mem`
  - Searches the value in the second operand for the first set bit from the LSB
  - If found, clears ZF and stores the index of the bit in the destination register
  - If the second operand is zero, sets ZF and does not change *destreg*
- `BSR` *`destreg`*`, reg/mem`
  - Searches for the first MSB set

©2024 Pavel Yosifovich

Exchange Instruction

- `XCHG reg/mem, reg/mem`
- Uses the `LOCK` prefix to make the exchange multiprocessor-safe if one of the operands is memory
  - Two memory operands are not supported
- `XCHG EAX, EAX`
  - In legacy x86 used as a one-byte `NOP` (0x90)
  - In x64, 0x90 is treated specially to preserve this behavior
    - Use `NOP` in MASM/NASM

©2024 Pavel Yosifovich

Byte Swap

- `BSWAP reg32/64`
  - Converts the register from big endian to little endian or vice versa
  - E.g., `BSWAP EAX` exchanges bits 0:7 with 24:31 and bits 8:15 and bits 16:23
- Using a 16-bit register is undefined behavior
  - To convert a 16-bit register, use `XCHG` with the low and high bytes
    - E.g., `XCHG AL, AH`

©2024 Pavel Yosifovich

I/O Instructions

- The x86 architecture support a 16-bit I/O space
  - Distinct from memory space
- Accessing I/O ports

  `IN reg, port`

  `OUT port, reg`
  - *reg* must be AL, AX or EAX
  - *port* can be an 8-bit immediate value (0-255) ,or the DX register (allowing full access to the 16-bit I/O space)
- String I/O (see the docs)
  - INS/B/W/D, OUTS/B/W/D

©2024 Pavel Yosifovich

Summary

- Procedures
- Stack Frames
- Multiplication and Division
- String Instructions
- Bit Manipulation

©2024 Pavel Yosifovich